



UNIVERSITÀ DI PARMA



# Informatica e Laboratorio di Programmazione

*The true sign of intelligence is not knowledge  
but imagination.*

Albert Einstein

- $\sim \frac{2}{3}$  in Class (slides + code examples)
- $\sim \frac{1}{3}$  in Laboratory (actual programming)
- Slides, code examples, and laboratory assignments
  - <https://elly2025.didattica.unipr.it>
  - <https://www.ce.unipr.it/didattica/informatica/>
  - Warning: they are continuously updated!

- September 18<sup>th</sup> – December 17<sup>th</sup>
  - Monday 13:30-15:30 (Lab. E c/o Q02)
  - Wednesday 10:30-12:30 (Aula F c/o sede didattica ing.)
  - Thursday 10:30-12:30 (Aula F c/o sede didattica ing.)

- Pauses
  - October 23<sup>th</sup> (maybe)

- Candidates should
  - know basic concepts of Information Technology
  - be able to use PCs and deal with files
  - know how to read and write....

- Theory
  - Class
- Practical part AKA programming
  - Class + Laboratory
- Problem solving
  - Class + Laboratory

- Theory (Class)
- Basic information needed to understand how to program
  - Basic computer architecture
  - Algorithms
  - How numbers can be managed in a computer

- Programming part (class + laboratory)
  - Centered on the programming language and algorithms
  - C fundamentals
  - Variables
  - Control instructions
  - Expressions
  - Array, strings & pointers
  - Functions and recursion
  - I/O
- Slides + code examples



- How to access laboratories
- You **must** use your official UNIPR account:
  - name.surnameXY@studenti.unipr.it
- Temporary you can use “exam” accounts:
  - User: workstation name, Password: “noncopiare”
- Feel free to use your laptop/notebook
  - Caveat: during the exam you can not do that!

- Problem solving (class + laboratory)
- Laboratory assignments and old exam for practicing
  - live

- C programming in laboratory
  - Initially we will discuss the assignment all together
  - ~2 hours
  - C
  - Modular approach
- Rating
  - No errors (and also warnings) when compiling
  - It must be executable
  - Please, try to pass the exam when you are ready

- <https://www.ce.unipr.it/didattica/informatica/esami/>



**UNIVERSITÀ  
DI PARMA**

Correzioni online di Informatica & Laboratorio di Programmazione

Menù

Da queste pagine potete accedere alle correzioni online delle prove pratiche di Informatica & Laboratorio di Programmazione  
[Ritorna](#) alla pagina del corso.

## Elenco prove pratiche:

### A.A. 2023-2024

1. [18 settembre 2024](#) *Allocazione dinamica di array bidimensionali, battaglia navale*
2. [2 settembre 2024](#) *Memorizzazione dinamica di coppie di stringhe, palindromi e anagrammi*
3. [31 luglio 2024](#) *Problema di Milena, lettura da file ASCII con righe con lunghezze differenti, calcolo massimo*
4. [17 giugno 2024](#) *Lettura, memorizzazione e somma polinomi. Array bidimensionali di numeri int*
5. [4 aprile 2024](#) *Allocazione dinamica di un vettore, calcolo numero divisori primi*
6. [19 febbraio 2024](#) *Controllo validità quadrati magici, allocazione dinamica matrici*
7. [15 gennaio 2024](#) *Lettura file CSV in array di struct, ordinamento con qsort()*
8. [21 dicembre 2023](#) *Lettura di file contenenti cruciverba, array bidimensionali di char, conversione di array di char in stringhe*

### A.A. 2022-2023

1. [15 settembre 2023](#) *Lettura file con array di stringhe, creazione stringa di codifica*
2. [4 settembre 2023](#) *Problema di Giuseppe, analisi array e relative allocazioni*



- Do not underestimate the effort needed to pass the exam
- Usually students that do not attend lessons → negative result
- Some statistics (2023 cohort)
  - Passed: 52.2% (48 students out of 102)
  - Not Passed: 22.8% (21 students out of 102)
  - Time average: 1.9 months
  - Average n. of attempts: 1.8
  - Average result: 24.01

# How to approach this course



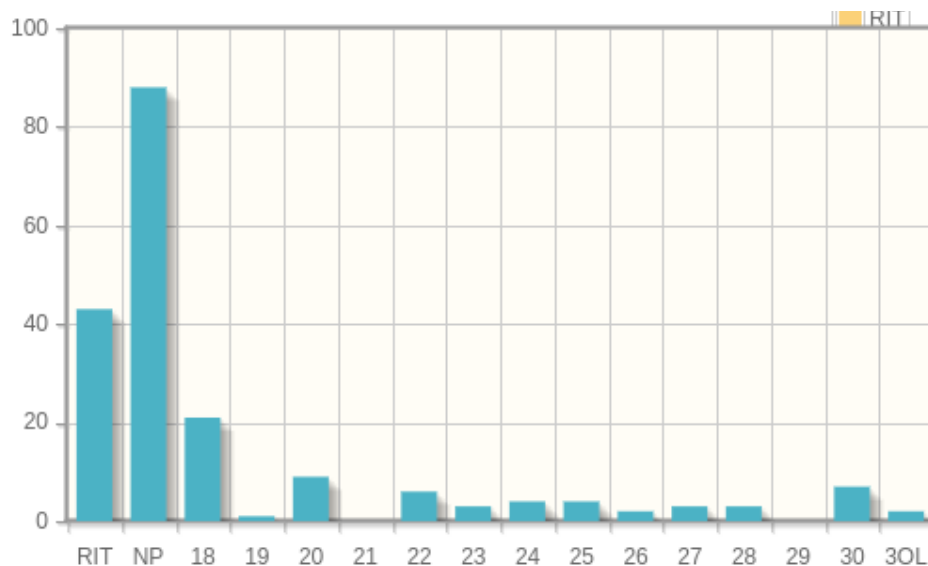
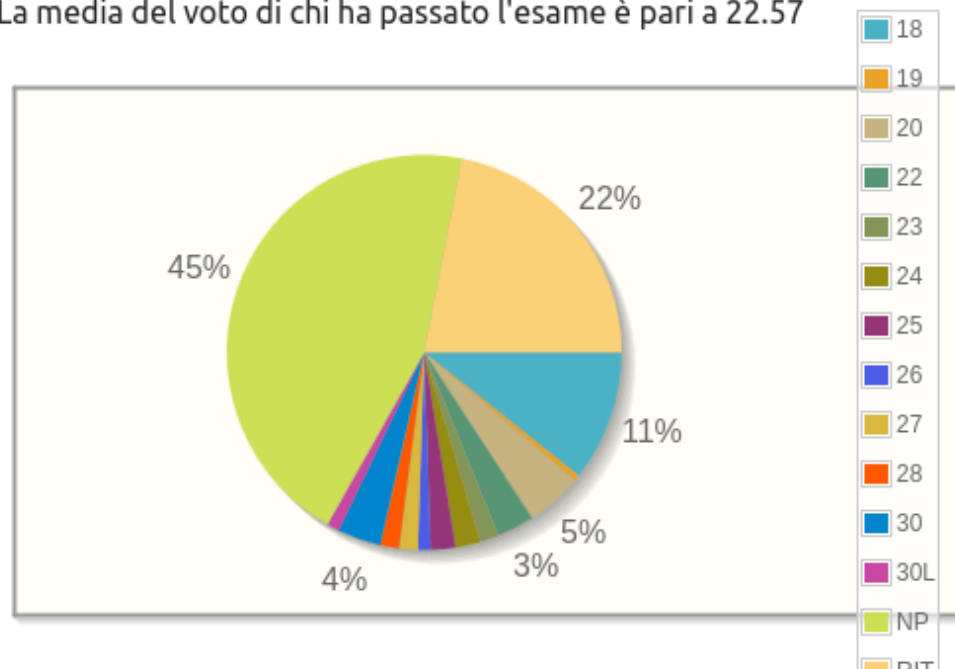
- Regular → less than 3 attempts, before 6 months from 1st registration, and result  $< 27$
- Brilliant → less than 3 attempts, before 6 months from 1st registration, and result  $\geq 27$
- Demanding → more than 3 registrations  $\geq 27$
- Difficult → more than 3 registrations  $< 27$
- Planned → less than 3 registrations but after 6 months

# How to approach this course

## Statistiche dei risultati dall'A.A. 2023-2024

Hanno passato la prova: 65 su 153 con un tasso pari a 42.48% oppure pari a 33.16% considerando anche i ritirati

La media del voto di chi ha passato l'esame è pari a 22.57



- Tuesday afternoon (tentatively) > 16:00
- Bldg. 1 of Cubes (scientific venue of engineering), 1<sup>st</sup> floor





- You can download
  - Slides we discuss during lessons
  - C examples we use with slides
  - Assignments we use in laboratory + code
- Slides & Code examples are mostly definitive (besides small corrections)
- Assignments → continuously updated

- How to download it?
  - Github!
  - Via web
  - Using “git” o “repo”
- Detailed instructions at:
  - <https://www.ce.unipr.it/didattica/informatica/repo.php>

- Good C manual
- Suggestions:
  - K.N. King, *C Programming: A Modern Approach (2<sup>nd</sup> Edition)*, W W Norton & Co
  - Bellini Guidi, *Linguaggio C*, Mc Graw Hill
  - Darnell Margolis, *C manuale di programmazione*, Mc Graw Hill



|                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                          |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|
| <i>Returns</i>  | c (the character written). If a write error occurs, putchar sets the stream's error indicator and returns EOF.                                                                                                                                                                                                                                                                                                                                                                                       | 7.3, 22.4                |
| <b>puts</b>     | <i>Write String</i><br><code>int puts(const char *s);</code><br>Writes the string pointed to by s to the stdout stream, then writes a new-line character.                                                                                                                                                                                                                                                                                                                                            | <stdio.h>                |
| <i>Returns</i>  | A nonnegative value if successful. Returns EOF if a write error occurs.                                                                                                                                                                                                                                                                                                                                                                                                                              | 13.3, 22.5               |
| <b>putwc</b>    | <i>Write Wide Character to File (C99)</i><br><code>wint_t putwc(wchar_t c, FILE *stream);</code><br>Wide-character version of putc.                                                                                                                                                                                                                                                                                                                                                                  | <wchar.h><br>25.5        |
| <b>putwchar</b> | <i>Write Wide Character (C99)</i><br><code>wint_t putwchar(wchar_t c);</code><br>Wide-character version of putchar.                                                                                                                                                                                                                                                                                                                                                                                  | <wchar.h><br>25.5        |
| <b>qsort</b>    | <i>Sort Array</i><br><code>void qsort(void *base, size_t nmemb, size_t size,<br/>int (*compar)(const void *, const void *));</code><br>Sorts the array pointed to by base. The array has nmemb elements, each size bytes long. compar is a pointer to a comparison function. When passed pointers to two array elements, the comparison function must return a negative, zero, or positive integer, depending on whether the first array element is less than, equal to, or greater than the second. | <stdlib.h><br>17.7, 26.2 |
| <b>raise</b>    | <i>Raise Signal</i><br><code>int raise(int sig);</code><br>Raises the signal whose number is sig.                                                                                                                                                                                                                                                                                                                                                                                                    | <signal.h>               |
| <i>Returns</i>  | Zero if successful, nonzero otherwise.                                                                                                                                                                                                                                                                                                                                                                                                                                                               | 24.3                     |
| <b>rand</b>     | <i>Generate Pseudo-Random Number</i><br><code>int rand(void);</code>                                                                                                                                                                                                                                                                                                                                                                                                                                 | <stdlib.h>               |
| <i>Returns</i>  | A pseudo-random integer between 0 and RAND_MAX (inclusive).                                                                                                                                                                                                                                                                                                                                                                                                                                          | 26.2                     |
| <b>realloc</b>  | <i>Resize Memory Block</i><br><code>void *realloc(void *ptr, size_t size);</code><br>ptr is assumed to point to a block of memory previously obtained from calloc, malloc, or realloc. realloc allocates a block of size bytes, copying the contents of the old block if necessary.                                                                                                                                                                                                                  | <stdlib.h>               |
| <i>Returns</i>  | A pointer to the beginning of the new memory block. Returns a null pointer if a block of the requested size can't be allocated.                                                                                                                                                                                                                                                                                                                                                                      | 17.3                     |



## A.14.2 Pseudo-Random Number Generator Functions

The *rand()* and *srand()* functions enable you to generate pseudo-random numbers.

### A.14.2.1 The *rand()* Function

```
#include <stdlib.h>
int rand( void );
```

The *rand()* function returns an integer in the range 0 through `RAND_MAX`. Successive calls to *rand()* should produce different integers. However, the sequence of random numbers could be the same for each program execution unless you use a different seed value via the *srand()* function.

### A.14.2.2 The *srand()* Function

```
#include <stdlib.h>
void srand( unsigned int seed );
```

The *srand()* function uses the argument as a seed for a new sequence of pseudo-random numbers to be returned by subsequent calls to *rand()*. If *srand()* is invoked with the same seed value, the sequence of generated numbers will be the same. The default seed value is 1.

## A.14.3 Memory Management Functions

The memory management functions enable you to allocate and deallocate memory dynamically. See Chapter 7 for more information about these functions.

### A.14.3.1 The *calloc()* Function

```
#include <stdlib.h>
void *calloc( size_t nmemb, size_t size );
```

The *calloc()* function allocates contiguous space for *nmemb* objects, each of which has a length in bytes specified by *size*. All bits in the allocated space are initialized to zero. *calloc()* returns a pointer to the first byte of the allocated space. If the space cannot be allocated, or if *nelem* or *size* is zero, *calloc()* returns a null pointer.

- Massimo Bertozzi
- [bertozzi@ce.unipr.it](mailto:bertozzi@ce.unipr.it)
- 0521 90 5845



UNIVERSITÀ DI PARMA

# Informatica e Laboratorio di Programmazione



KEEP  
CALM  
IT'S  
QUESTION  
TIME

*nec scire fas est omnia*

*Quinto Orazio Flacco, Odi*







- $\sim \frac{2}{3}$  in Class (slides + code examples)
- $\sim \frac{1}{3}$  in Laboratory (actual programming)
- Slides, code examples, and laboratory assignments
  - <https://elly2025.didattica.unipr.it>
  - <https://www.ce.unipr.it/didattica/informatica/>
  - Warning: they are continuously updated!



- September 18<sup>th</sup> – December 17<sup>th</sup>
  - Monday 13:30-15:30 (Lab. E c/o Q02)
  - Wednesday 10:30-12:30 (Aula F c/o sede didattica ing.)
  - Thursday 10:30-12:30 (Aula F c/o sede didattica ing.)



- Pauses
  - October 23<sup>th</sup> (maybe)



- Candidates should
  - know basic concepts of Information Technology
  - be able to use PCs and deal with files
  - know how to read and write....

- Theory
  - Class
- Practical part AKA programming
  - Class + Laboratory
- Problem solving
  - Class + Laboratory



- Theory (Class)
- Basic information needed to understand how to program
  - Basic computer architecture
  - Algorithms
  - How numbers can be managed in a computer



- Programming part (class + laboratory)
  - Centered on the programming language and algorithms
  - C fundamentals
  - Variables
  - Control instructions
  - Expressions
  - Array, strings & pointers
  - Functions and recursion
  - I/O
- Slides + code examples



- How to access laboratories
- You **must** use your official UNIPR account:
  - name.surnameXY@studenti.unipr.it
- Temporary you can use “exam” accounts:
  - User: workstation name, Password: “noncopiare”
- Feel free to use your laptop/notebook
  - Caveat: during the exam you can not do that!

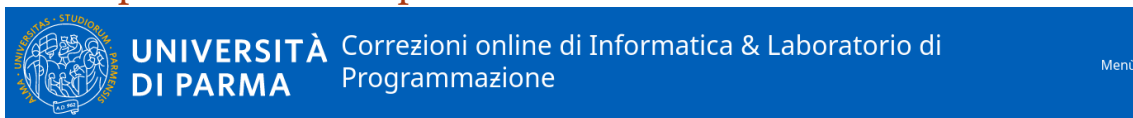


- Problem solving (class + laboratory)
- Laboratory assignments and old exam for practicing
  - live



- C programming in laboratory
  - Initially we will discuss the assignment all together
  - ~2 hours
  - C
  - Modular approach
- Rating
  - No errors (and also warnings) when compiling
  - It must be executable
  - Please, try to pass the exam when you are ready

- <https://www.ce.unipr.it/didattica/informatica/esami/>



Da queste pagine potete accedere alle correzioni online delle prove pratiche di Informatica & Laboratorio di Programmazione  
[Ritorna](#) alla pagina del corso.

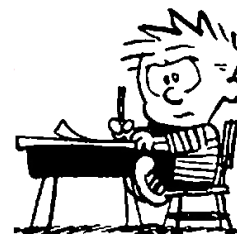
## Elenco prove pratiche:

### A.A. 2023-2024

- |                      |                                                                                                               |
|----------------------|---------------------------------------------------------------------------------------------------------------|
| 1. 18 settembre 2024 | Allocazione dinamica di array bidimensionali, battaglia navale                                                |
| 2. 2 settembre 2024  | Memorizzazione dinamica di coppie di stringhe, palindromi e anagrammi                                         |
| 3. 31 luglio 2024    | Problema di Milena, lettura da file ASCII con righe con lunghezze differenti, calcolo massimo                 |
| 4. 17 giugno 2024    | Lettura, memorizzazione e somma polinomi. Array bidimensionali di numeri int                                  |
| 5. 4 aprile 2024     | Allocazione dinamica di un vettore, calcolo numero divisori primi                                             |
| 6. 19 febbraio 2024  | Controllo validità quadrati magici, allocazione dinamica matrici                                              |
| 7. 15 gennaio 2024   | Lettura file CSV in array di struct, ordinamento con qsort()                                                  |
| 8. 21 dicembre 2023  | Lettura di file contenenti cruciverba, array bidimensionali di char, conversione di array di char in stringhe |

### A.A. 2022-2023

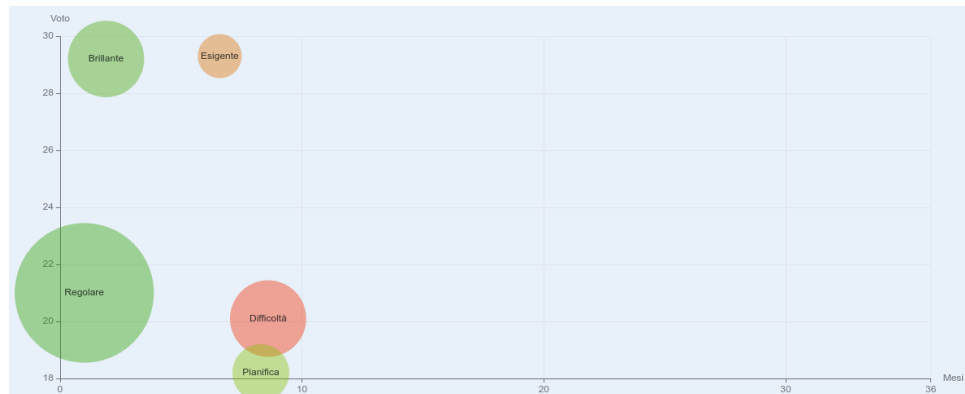
- |                      |                                                                   |
|----------------------|-------------------------------------------------------------------|
| 1. 15 settembre 2023 | Lettura file con array di stringhe, creazione stringa di codifica |
| 2. 4 settembre 2023  | Problema di Giuseppe, analisi array e relative allocazioni        |





- Do not underestimate the effort needed to pass the exam
- Usually students that do not attend lessons → negative result
- Some statistics (2023 cohort)
  - Passed: 52.2% (48 students out of 102)
  - Not Passed: 22.8% (21 students out of 102)
  - Time average: 1.9 months
  - Average n. of attempts: 1.8
  - Average result: 24.01

# How to approach this course



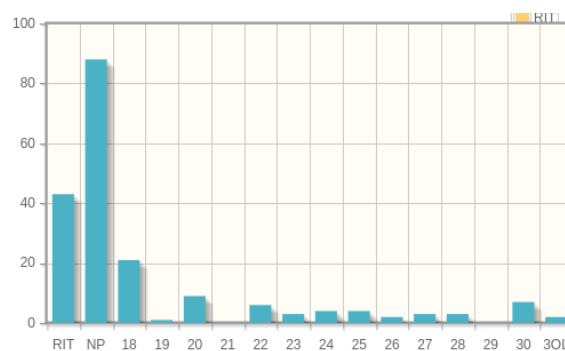
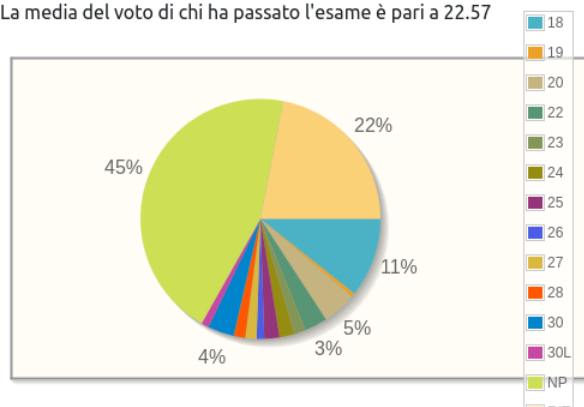
- Regular → less than 3 attempts, before 6 months from 1st registration, and result < 27
- Brilliant → less than 3 attempts, before 6 months from 1st registration, and result  $\geq 27$
- Demanding → more than 3 registrations  $\geq 27$
- Difficult → more than 3 registrations < 27
- Planned → less than 3 registrations but after 6 months



## Statistiche dei risultati dall'A.A. 2023-2024

Hanno passato la prova: 65 su 153 con un tasso pari a 42.48% oppure pari a 33.16% considerando anche i ritirati

La media del voto di chi ha passato l'esame è pari a 22.57



- Tuesday afternoon (tentatively) > 16:00
- Bldg. 1 of Cubes (scientific venue of engineering), 1<sup>st</sup> floor





- You can download
  - Slides we discuss during lessons
  - C examples we use with slides
  - Assignments we use in laboratory + code
- Slides & Code examples are mostly definitive (besides small corrections)
- Assignments → continuously updated





- How to download it?
  - Github!
  - Via web
  - Using “git” o “repo”
- Detailed instructions at:
  - <https://www.ce.unipr.it/didattica/informatica/repo.php>



- Good C manual
- Suggestions:
  - K.N. King, *C Programming: A Modern Approach (2<sup>nd</sup> Edition)*, W W Norton & Co
  - Bellini Guidi, *Linguaggio C*, Mc Graw Hill
  - Darnell Margolis, *C manuale di programmazione*, Mc Graw Hill



|                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                          |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|
| <i>Returns</i>  | c (the character written). If a write error occurs, putchar sets the stream's error indicator and returns EOF.                                                                                                                                                                                                                                                                                                                                                                         | 7.3, 22.4                |
| <b>puts</b>     | <b>Write String</b><br>int puts(const char *s);<br>Writes the string pointed to by s to the stdout stream, then writes a new-line character.                                                                                                                                                                                                                                                                                                                                           | <stdio.h>                |
| <i>Returns</i>  | A nonnegative value if successful. Returns EOF if a write error occurs.                                                                                                                                                                                                                                                                                                                                                                                                                | 13.3, 22.5               |
| <b>putwc</b>    | <b>Write Wide Character to File (C99)</b><br>wint_t putwc(wchar_t c, FILE *stream);<br>Wide-character version of putc.                                                                                                                                                                                                                                                                                                                                                                 | <wchar.h><br>25.5        |
| <b>putwchar</b> | <b>Write Wide Character (C99)</b><br>wint_t putwchar(wchar_t c);<br>Wide-character version of putchar.                                                                                                                                                                                                                                                                                                                                                                                 | <wchar.h><br>25.5        |
| <b>qsort</b>    | <b>Sort Array</b><br>void qsort(void *base, size_t nmemb, size_t size,<br>int (*compar)(const void *, const void *));<br>Sorts the array pointed to by base. The array has nmemb elements, each size bytes long. compar is a pointer to a comparison function. When passed pointers to two array elements, the comparison function must return a negative, zero, or positive integer, depending on whether the first array element is less than, equal to, or greater than the second. | <stdlib.h><br>17.7, 26.2 |
| <b>raise</b>    | <b>Raise Signal</b><br>int raise(int sig);<br>Raises the signal whose number is sig.                                                                                                                                                                                                                                                                                                                                                                                                   | <signal.h>               |
| <i>Returns</i>  | Zero if successful, nonzero otherwise.                                                                                                                                                                                                                                                                                                                                                                                                                                                 | 24.3                     |
| <b>rand</b>     | <b>Generate Pseudo-Random Number</b><br>int rand(void);<br>A pseudo-random integer between 0 and RAND_MAX (inclusive).                                                                                                                                                                                                                                                                                                                                                                 | <stdlib.h><br>26.2       |
| <b>realloc</b>  | <b>Resize Memory Block</b><br>void *realloc(void *ptr, size_t size);<br>ptr is assumed to point to a block of memory previously obtained from calloc, malloc, or realloc. realloc allocates a block of size bytes, copying the contents of the old block if necessary.                                                                                                                                                                                                                 | <stdlib.h>               |
| <i>Returns</i>  | A pointer to the beginning of the new memory block. Returns a null pointer if a block of the requested size can't be allocated.                                                                                                                                                                                                                                                                                                                                                        | 17.3                     |



### A.14.2 Pseudo-Random Number Generator Functions

The *rand()* and *srand()* functions enable you to generate pseudo-random numbers.

#### A.14.2.1 The *rand()* Function

```
#include <stdlib.h>
int rand( void );
```

The *rand()* function returns an integer in the range 0 through *RAND\_MAX*. Successive calls to *rand()* should produce different integers. However, the sequence of random numbers could be the same for each program execution unless you use a different seed value via the *srand()* function.

#### A.14.2.2 The *srand()* Function

```
#include <stdlib.h>
void srand( unsigned int seed );
```

The *srand()* function uses the argument as a seed for a new sequence of pseudo-random numbers to be returned by subsequent calls to *rand()*. If *srand()* is invoked with the same seed value, the sequence of generated numbers will be the same. The default seed value is 1.

### A.14.3 Memory Management Functions

The memory management functions enable you to allocate and deallocate memory dynamically. See Chapter 7 for more information about these functions.

#### A.14.3.1 The *calloc()* Function

```
#include <stdlib.h>
void *calloc( size_t nmemb, size_t size );
```

The *calloc()* function allocates contiguous space for *nmemb* objects, each of which has a length in bytes specified by *size*. All bits in the allocated space are initialized to zero. *calloc()* returns a pointer to the first byte of the allocated space. If the space cannot be allocated, or if *nelem* or *size* is zero, *calloc()* returns a null pointer.

- Massimo Bertozzi
- [bertozzi@ce.unipr.it](mailto:bertozzi@ce.unipr.it)
- 0521 90 5845

